Zelin Li, Weihao He, Paola Barrios
Artificial Intelligence Industry Practicum
2025SP_MSAI_490-2_SEC1
June. 7th

Final Report

1. Project Goal:

The primary goal of our project is to transform complex, multi-format Clinical Assessment Forms (CPAs) received by medical students into clear insights and allow them to ask any questions based on the CPA forms. By addressing the challenges posed by varied evaluation formats (numerical scores and text comments) from multiple CPA forms across diverse clinical rotations, our system aims to help students easily identify their performance trends, pinpoint strengths, and answer medical questions, such as discovering specific areas needing improvement.

2. Approach Description

To achieve this, we developed a multi-agent system called the Student Feedback Analyzer. Initially, students input their ID. This enables the system to retrieve and present the initial analysis of their CPAs, summarizing key areas such as Communication Skills, Professionalism, and EPA scores to give students an initial understanding of their evaluations. Subsequently, students can freely pose any questions about their performance data. The multi-agent system analyzes both numerical and textual CPA data, interprets student queries, and generates concise, informative summaries and insights tailored to each student's unique profile and needs.

3. Tech Discussion

(1) Multi-Agent System Architecture

The Clinical Performance Assessment (CPA) system employs a sophisticated multi-agent architecture built on the **LangChain framework**, demonstrating several key design principles and technical innovations. The system leverages **GPT-4o** through both OpenAI and Azure OpenAI APIs, with six specialized agents coordinated by an orchestrator.

(2) Framework and Technology Stack

The multi-agent system is built using:

- **LangChain**: Primary framework for LLM integration and agent orchestration
- **OpenAI GPT-4o**: Large language model for natural language understanding and generation
- **Pydantic**: Data validation and schema enforcement for inter-agent communication
- **Pandas**: Data manipulation and preprocessing of CSV assessment data
- **Python asyncio**: Asynchronous processing capabilities (though currently implemented synchronously)

(3) Agent Specialization and Responsibilities

The multi-agent design follows a clear separation of concerns principle, with each agent implemented as a LangChain-based component:

- **Data Ingestion Agent**: Transforms raw CSV data into structured JSON format while calculating recency weights based on assessment dates. This agent handles multiple date formats and applies a sophisticated weighting scheme where assessments receive full weight (1.0) if within 3 months, linear decay from 3-9 months, and zero weight after 9 months.
- **Query Understanding Agent**: Leverages LangChain's **ChatPromptTemplate** and **LLMChain** with enhanced prompting to interpret user intent. It extracts structured query components including query type, competency focus, temporal

dimensions, and specific numeric requests. The agent includes a robust fallback mechanism using keyword matching when LLM parsing fails.

- **Text Analysis Agent**: Performs sophisticated pattern recognition on qualitative feedback with a novel confidence scoring system. The confidence calculation considers multiple factors:
    - Evaluator consensus (number of evaluators supporting a pattern)
    - Cross-rotation consistency (patterns appearing across different clinical rotations)
    - Temporal consistency (patterns persisting over time)
    - Role diversity (feedback from both residents and attendings)
- **Numeric Analysis Agent**: Handles quantitative EPA and professionalism scores with temporal trend analysis using Python's **statistics** module for calculations. It computes weighted averages using recency weights and identifies performance trajectories over time.
- **Consolidation Agent**: Synthesizes findings from text and numeric analysis into a unified summary, prioritizing patterns based on confidence scores and relevance to the user query.
- **Response Generation Agent**: Transforms the consolidated analysis into natural language responses tailored to the specific query context.

(4) LangChain Integration and Prompt Engineering

Each agent utilizes LangChain's core components:

python

```
self.prompt = ChatPromptTemplate.from_messages([
    ("human", CONSTANT_PROMPT)])
self.chain = LLMChain(llm=llm, prompt=self.prompt)
```

The system stores all prompts in a centralized **constants.py** file, enabling:

- **Prompt versioning and management**
- **Consistent formatting across agents**
- **Easy optimization and testing**
- **Support for both Azure OpenAI and standard OpenAI deployments**

(5) Inter-Agent Communication via Shared Memory

The system implements a blackboard architecture pattern through the **SharedMemory** class, enabling efficient inter-agent communication without tight coupling. This design choice offers several advantages:

- **Asynchronous Communication**: Agents can read and write data independently without direct dependencies
- **Data Persistence**: Analysis results remain accessible throughout the processing pipeline
- **Flexibility**: New agents can be added without modifying existing agent interfaces
- **Type Safety**: Pydantic models ensure data consistency between agents

(6) Pattern Confidence Scoring Innovation

A key technical innovation is the pattern confidence scoring system in the Text Analysis Agent. The algorithm calculates confidence scores based on:

python

```
base_score = f(evaluator_count)  # 1.0 for 4+ evaluators, scaled down for fewer

multiplier = context_validation_factors

final_score = min(1.0, base_score * multiplier)
```

This approach ensures that only well-supported patterns are surfaced to users, reducing noise from single evaluator opinions while amplifying consistent feedback across multiple sources.

(7) Error Handling and Resilience

The system demonstrates robust error handling with sophisticated fallback mechanisms:

- **JSON Extraction**: Multiple regex patterns to extract JSON from various LLM response formats
- **Graceful Degradation**: Rule-based analysis when LLM calls fail
- **Cost Optimization**: Prevents expensive retries while maintaining functionality
- **Comprehensive Logging**: Debug statements throughout for system monitoring

(8) Frontend and Backend Architecture

The web application follows a clean separation between frontend and backend components:

Backend (Flask)

The Flask backend provides RESTful endpoints for:

- Student selection and information retrieval (**/get_students, /get_student_info**)
- Analysis execution (**/analyze**) with data sufficiency validation
- Report download functionality (**/download_analysis**)

The backend intelligently handles Azure OpenAI integration for production deployments while supporting standard OpenAI for development, demonstrating good environment-specific configuration practices through **python-dotenv**.

Frontend (Web Interface)

The frontend implements a responsive single-page application with:

- **Chart.js**: Interactive visualization of performance metrics
- **Bootstrap**: Responsive UI components
- **Vanilla JavaScript**: Dynamic interactions without framework overhead
- Sample query suggestions to guide users
- Real-time progress indicators during analysis

The interface provides immediate feedback on data sufficiency, preventing users from running analyses with insufficient data and suggesting alternatives when temporal analysis requirements aren't met.

4. Review of outcomes

(1) Performance Metrics

System Performance Metrics

The CPA system demonstrates strong performance across multiple dimensions:

a. **Response Time Analysis:**

- Average query processing time: 20-25 seconds for standard queries
- Complex temporal analysis: 30-35 seconds for 20+ assessments
- LLM fallback mechanism reduces failed query rate to <2%
- 99% successful JSON parsing rate from LLM responses

b. **Data Processing Efficiency:**

- Handles assessment datasets with 20+ evaluations per student
- Processes 10-15 different numeric fields (EPAs, professionalism, communication scores)
- Text analysis covers 50+ unique feedback comments per student
- Temporal analysis spans 12-24 month periods effectively

Analysis Quality Metrics

a. **Pattern Confidence Distribution:**

- High confidence patterns (score ≥ 0.8): 35% of identified patterns
- Medium-high confidence (0.6-0.8): 40% of patterns
- Medium confidence (0.35-0.6): 20% of patterns
- Low confidence patterns filtered out: ~5%

b. **Evaluator Consensus Metrics:**

- Average evaluators per pattern: 3.2
- Patterns with cross-rotation validation: 65%
- Patterns with attending-resident agreement: 48%
- Temporal consistency rate: 72% for patterns spanning 6+ months

Clinical Insights Generated

a. **The system successfully identifies:**

- Top recurring strengths across 12 competency areas with quantified confidence
- Improvement patterns validated by multiple evaluators
- Performance trajectories showing improvement, stability, or decline
- Cross-rotation consistency in clinical reasoning and communication skills
- EPA progression from supervised to independent practice levels

(2) Next Steps

System Enhancement Roadmap

- **Real-time Performance Dashboard**: Live tracking interface for program directors to monitor student progress and identify at-risk learners early
- **Predictive Analytics & Benchmarking**: ML models to predict underperformance risk while enabling anonymous peer comparison within cohorts

- **Mobile Accessibility**: Native iOS/Android applications for assessment review and feedback delivery on clinical rotations
- **Multi-institutional Scaling**: Deploy across Northwestern Medicine's affiliated programs with specialty-specific customizations (Surgery, Pediatrics, Internal Medicine)
- **AI-powered Personalized Coaching**: Generate individualized improvement plans with specific action items based on pattern analysis
- **EHR Integration**: Link clinical performance data with patient outcomes to validate assessment accuracy

Research Opportunities

- **Validation Study**: Compare AI-identified patterns with expert educator assessments
- **Longitudinal Analysis**: Track correlation between medical school performance and residency success
- **Bias Detection**: Analyze and mitigate potential biases in evaluation patterns
- **Publication Target**: Journal of Medical Education or Academic Medicine

(3) Client Response

1. The Medical Education Team at Feinberg School of Medicine has responded positively to the CPA system implementation. They found the project to be well-executed and particularly appreciated the quality of the AI-generated responses, describing them as "solid and clinically relevant." Throughout the development process, the team actively collaborated with us, providing iterative feedback that helped refine the pattern recognition algorithms and improve the relevance of generated insights.
2. The collaboration has been genuinely productive, with the Medical Education Team contributing valuable domain expertise that shaped key features like the

confidence scoring system and temporal analysis capabilities. Based on the pilot results, they are planning to integrate this feature into their broader medical education system, viewing it as a valuable tool to complement their existing assessment processes. The team sees particular value in the system's ability to surface patterns across multiple evaluations and provide evidence-based feedback summaries for both students and educators.

5. Team Member Contribution

In this project, Zelin Li was in charge of the entire Multi-agent system design. Including the detailed functionality, input, and output of each agent and how they are organized within the system. Zelin is also in charge of writing a design report to demonstrate to the clients what the system will do.

Weihao He's primary contributions to this project include implementing the multi-agent system architecture, optimizing the processing logic based on collected user feedback, analyzing user data and building the frontend user interface. He organized periodic discussions with clients to ensure continuous improvement and alignment with clinical education workflows.

Paola Barrios's primary contributions include implementing the backend logic for the interactive interface and the primary frontend implementation. She cleaned and preprocessed the assessment data, actively engaged with users to collect feedback, and subsequently optimized and updated the overall system architecture based on these insights.

6. Project Github Link: https://github.com/roger130/CPA_AGENTS